```c
// Comments marked by initials ER, KP, KL, DS, EC
//###########################################################################
// FILE:   FinalProjectStarter_main.c
//
// TITLE:  Final Project Starter
//###########################################################################

// Included Files
#include <stdio.h>
#include <stdlib.h>
#include <stdarg.h>
#include <string.h>
#include <math.h>
#include <limits.h>
#include "F28x_Project.h"
#include "F2837xD_SWPrioritizedIsrLevels.h"
#include "driverlib.h"
#include "device.h"
#include "F28379dSerial.h"
#include "song.h"
#include "dsp.h"
#include "fpu32/fpu_rfft.h"
#include "xy.h"
#include "MatrixMath.h"
#include "SE423Lib.h"
#include "OptiTrack.h"

#define PI          3.14159265358979323846264433832795
#define TWOPI       6.28318530717958647692528676655 9
#define HALFPI      1.57079632679489661923132169163 98
// The Launchpad's CPU Frequency set to 200 you should not change this value
#define LAUNCHPAD_CPU_FREQUENCY 200

// ----- code for CAN start here -----
#include "F28379dCAN.h"
//#define TX_MSG_DATA_LENGTH    4
//#define TX_MSG_OBJ_ID         0  //transmit

#define RX_MSG_DATA_LENGTH    8
#define RX_MSG_OBJ_ID_1       1  //measurement from sensor 1
#define RX_MSG_OBJ_ID_2       2  //measurement from sensor 2
#define RX_MSG_OBJ_ID_3       3  //quality from sensor 1
#define RX_MSG_OBJ_ID_4       4  //quality from sensor 2
// ----- code for CAN end here -----

// Interrupt Service Routines predefinition
__interrupt void cpu_timer0_isr(void);
__interrupt void cpu_timer1_isr(void);
__interrupt void cpu_timer2_isr(void);
__interrupt void SWI1_HighestPriority(void);
__interrupt void SWI2_MiddlePriority(void);
__interrupt void SWI3_LowestPriority(void);
__interrupt void ADCC_ISR(void);
__interrupt void SPIB_isr(void);
// ----- code for CAN start here -----
__interrupt void can_isr(void);
// ----- code for CAN end here -----

void setF28027EPWM1A(float controleffort);
int16_t EPwm1A_F28027 = 1500;
void setF28027EPWM2A(float controleffort);
int16_t EPwm2A_F28027 = 1500;


// ----- code for CAN start here -----
// volatile uint32_t txMsgCount = 0;
// extern uint16_t txMsgData[4];

volatile uint32_t rxMsgCount_1 = 0;
volatile uint32_t rxMsgCount_2 = 0;
extern uint16_t rxMsgData[8];

uint32_t dis_raw_1[2];
uint32_t dis_raw_2[2];
uint32_t dis_1 = 0;
uint32_t dis_2 = 0;

uint32_t quality_raw_1[4];
uint32_t quality_raw_2[4];
float quality_1 = 0.0;
float quality_2 = 0.0;

uint32_t lightlevel_raw_1[4];
uint32_t lightlevel_raw_2[4];
float lightlevel_1 = 0.0;
float lightlevel_2 = 0.0;

uint32_t measure_status_1 = 0;
uint32_t measure_status_2 = 0;

volatile uint32_t errorFlag = 0;
// ----- code for CAN end here -----

uint32_t numTimer0calls = 0;
uint16_t UARTPrint = 0;

float printLV1 = 0;
float printLV2 = 0;

float printLinux1 = 0;
float printLinux2 = 0;

uint16_t LADARi = 0;
char G_command[] = "G04472503\n"; //command for getting distance -120 to 120 degree
uint16_t G_len = 11; //length of command
xy ladar_pts[228]; //xy data
float LADARrightfront = 0;
float LADARfront = 0;
float LADARtemp_x = 0;
float LADARtemp_y = 0;
extern datapts ladar_data[228];

extern uint16_t newLinuxCommands;
extern float LinuxCommands[CMDNUM_FROM_FLOATS];

extern uint16_t NewLVData;
extern float fromLVvalues[LVNUM_TOFROM_FLOATS];
extern LVSendFloats_t DataToLabView;
extern char LVsenddata[LVNUM_TOFROM_FLOATS*4+2];
extern uint16_t LADARpingpong;
extern uint16_t NewCAMDataThreshold1;  // Flag new data
extern float fromCAMvaluesThreshold1[CAMNUM_FROM_FLOATS];
extern uint16_t NewCAMDataThreshold2;  // Flag new data
extern float fromCAMvaluesThreshold2[CAMNUM_FROM_FLOATS];

extern uint16_t NewCAMDataAprilTag1;  // Flag new data
extern float fromCAMvaluesAprilTag1[CAMNUM_FROM_FLOATS];

float tagid = 0;
float tagx = 0;
float tagy = 0;
float tagz = 0;
float tagthetax = 0;
float tagthetay = 0;
float tagthetaz = 0;
int32_t numtag = 0;
int32_t AprilTagState = 10;
float KpAprilTag = -1.0;
float AprilTagSetPoint = 0;
int32_t AprilTagCount = 0;
```

```

float MaxAreaThreshold1 = 0;
float MaxColThreshold1 = 0;
float MaxRowThreshold1 = 0;
float NextLargestAreaThreshold1 = 0;
float NextLargestColThreshold1 = 0;
float NextLargestRowThreshold1 = 0;
float NextNextLargestAreaThreshold1 = 0;
float NextNextLargestColThreshold1 = 0;
float NextNextLargestRowThreshold1 = 0;

float MaxAreaThreshold2 = 0;
float MaxColThreshold2 = 0;
float MaxRowThreshold2 = 0;
float NextLargestAreaThreshold2 = 0;
float NextLargestColThreshold2 = 0;
float NextLargestRowThreshold2 = 0;
float NextNextLargestAreaThreshold2 = 0;
float NextNextLargestColThreshold2 = 0;
float NextNextLargestRowThreshold2 = 0;

uint32_t numThres1 = 0;
uint32_t numThres2 = 0;

pose ROBOTps = {0,0,0}; //robot position
pose LADARps = {3.5/12.0,0,1};  // 3.5/12 for front mounting, theta is not used in this current code
float LADARxoffset = 0;
float LADARyoffset = 0;

uint32_t timecount = 0;
int16_t RobotState = 1;
int16_t checkfronttally = 0;
int32_t WallFollowtime = 0;

#define NUMWAYPOINTS 10 //Modified NUMWAYPOINTS from 8 to 10 for two extra waypoint targets for the robot path - DS
uint16_t statePos = 0;
pose robotdest[NUMWAYPOINTS];  // array of waypoints for the robot
uint16_t i = 0;//for loop

uint16_t right_wall_follow_state = 2;  // right follow
float Kp_front_wall = -2.0;
float front_turn_velocity = 0.2;
float left_turn_Stop_threshold = 3.5;
float Kp_right_wal = -4.0;
float ref_right_wall = 1.1;
float foward_velocity = 1.0;
float left_turn_Start_threshold = 1.3;
float turn_saturation = 2.5;

//RC Servo
float Gate_O = -67.74;
float Gate_C = 0.40;
float RTong = -49.34; //KLEC: ball goes to the left
float MTong =  -11.49;
float LTong = 1.47; //KLEC: ball goes to the right

float x_pred[3][1] = {{0},{0},{0}};                // predicted state

//more kalman vars
float B[3][2] = {{1,0},{1,0},{0,1}};            // control input model
float u[2][1] = {{0},{0}};          // control input in terms of velocity and angular velocity
float Bu[3][1] = {{0},{0},{0}}; // matrix multiplication of B and u
float z[3][1];                              // state measurement
float eye3[3][3] = {{1,0,0},{0,1,0},{0,0,1}};   // 3x3 identity matrix
float K[3][3] = {{1,0,0},{0,1,0},{0,0,1}};      // optimal Kalman gain
#define ProcUncert 0.0001
#define CovScalar 10
float Q[3][3] = {{ProcUncert,0,ProcUncert/CovScalar},
                 {0,ProcUncert,ProcUncert/CovScalar},
                 {ProcUncert/CovScalar,ProcUncert/CovScalar,ProcUncert}};   // process noise (covariance of encoders and gyro)
#define MeasUncert 1
float R[3][3] = {{MeasUncert,0,MeasUncert/CovScalar},
                 {0,MeasUncert,MeasUncert/CovScalar},
                 {MeasUncert/CovScalar,MeasUncert/CovScalar,MeasUncert}};   // measurement noise (covariance of OptiTrack Motion Capture measurement)
float S[3][3] = {{1,0,0},{0,1,0},{0,0,1}};  // innovation covariance
float S_inv[3][3] = {{1,0,0},{0,1,0},{0,0,1}};  // innovation covariance matrix inverse
float P_pred[3][3] = {{1,0,0},{0,1,0},{0,0,1}}; // predicted covariance (measure of uncertainty for current position)
float temp_3x3[3][3];               // intermediate storage
float temp_3x1[3][1];               // intermediate storage
float ytilde[3][1];                 // difference between predictions

// Optitrack Variables
int32_t OptiNumUpdatesProcessed = 0;
pose OPTITRACKps;
extern uint16_t new_optitrack;
extern float Optitrackdata[OPTITRACKDATASIZE];
int16_t newOPTITRACKpose=0;

int16_t adcc2result = 0;
int16_t adcc3result = 0;
int16_t adcc4result = 0;
int16_t adcc5result = 0;
float adcC2Volt = 0.0;
float adcC3Volt = 0.0;
float adcC4Volt = 0.0;
float adcC5Volt = 0.0;
int32_t numADCCcalls = 0;
float LeftWheel = 0;
float RightWheel = 0;
float LeftWheel_1 = 0;
float RightWheel_1 = 0;
float LeftVel = 0;
float RightVel = 0;
float uLeft = 0;
float uRight = 0;
float HandValue = 0;
float vref = 0;
float turn = 0;
float gyro9250_drift = 0;
float gyro9250_angle = 0;
float old_gyro9250 = 0;
float gyroLPR510_angle = 0;
float gyroLPR510_offset = 0;
float gyroLPR510_drift = 0;
float old_gyroLPR510 = 0;
float gyro9250_radians = 0;
float gyroLPR510_radians = 0;

int16_t readdata[25];
int16_t IMU_data[9];

float accelx = 0;
float accely = 0;
float accelz = 0;
float gyrox = 0;
float gyroy = 0;
float gyroz = 0;

// Needed global Variables
float accelx_offset = 0;
float accely_offset = 0;
float accelz_offset = 0;
float gyrox_offset = 0;
float gyroy_offset = 0;
float gyroz_offset = 0;
int16_t calibration_state = 0;
int32_t calibration_count = 0;
int16_t doneCal = 0;

#define MPU9250 1
#define DAN28027 2
int16_t CurrentChip = MPU9250;
```

```
int16_t DAN28027Garbage = 0;
int16_t dan28027adc1 = 0;
int16_t dan28027adc2 = 0;
uint16_t MPU9250ignoreCNT = 0;  //This is ignoring the first few interrupts if ADCC_ISR and start sending to IMU after these first few interrupts.
//Added variables to store camera-based ball distances - DS
float robotToBall2 = 0;
float robotToBall1 = 0;

// angle of the hand encoder - ER
float RCangle = 0.0;
float colcentroid1 = 0.0;
float colcentroid2 = 0.0;
float kpvision = -0.07; // vision proportional gain chosen so robot turns toward the ball centroid without overcorrecting - DS

int16_t count = 0;
int16_t statecount = 2000;
int16_t dwell=0;
float current= 0.0;
float target = 0.0;

void main(void)
{
    // PLL, WatchDog, enable Peripheral Clocks
    // This example function is found in the F2837xD_SysCtrl.c file.
    InitSysCtrl();

    InitGpio();

    InitSE423DefaultGPIO();

    //Scope for Timing
    GPIO_SetupPinMux(11, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(11, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;

    GPIO_SetupPinMux(32, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(32, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;

    GPIO_SetupPinMux(52, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(52, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO52 = 1;

    GPIO_SetupPinMux(60, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(60, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO60 = 1;

    GPIO_SetupPinMux(61, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(61, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;

    GPIO_SetupPinMux(67, GPIO_MUX_CPU1, 0);
    GPIO_SetupPinOptions(67, GPIO_OUTPUT, GPIO_PUSHPULL);
    GpioDataRegs.GPCCLEAR.bit.GPIO67 = 1;

    // ----- code for CAN start here -----
    //GPIO17 - CANRXB
    GPIO_SetupPinMux(17, GPIO_MUX_CPU1, 2);
    GPIO_SetupPinOptions(17, GPIO_INPUT, GPIO_ASYNC);

    //GPIO12 - CANTXB
    GPIO_SetupPinMux(12, GPIO_MUX_CPU1, 2);
    GPIO_SetupPinOptions(12, GPIO_OUTPUT, GPIO_PUSHPULL);
    // ----- code for CAN end here -----



    // ----- code for CAN start here -----
    // Initialize the CAN controller
    InitCANB();

    // Set up the CAN bus bit rate to 1000 kbps
    setCANBitRate(200000000, 1000000);

    // Enables Interrupt line 0, Error & Status Change interrupts in CAN_CTL register.
    CanbRegs.CAN_CTL.bit.IE0= 1;
    CanbRegs.CAN_CTL.bit.EIE= 1;
    // ----- code for CAN end here -----

    // Clear all interrupts and initialize PIE vector table:
    // Disable CPU interrupts
    DINT;

    // Initialize the PIE control registers to their default state.
    // The default state is all PIE interrupts disabled and flags
    // are cleared.
    // This function is found in the F2837xD_PieCtrl.c file.
    InitPieCtrl();

    // Disable CPU interrupts and clear all CPU interrupt flags:
    IER = 0x0000;
    IFR = 0x0000;

    // Initialize the PIE vector table with pointers to the shell Interrupt
    // Service Routines (ISR).
    // This will populate the entire table, even if the interrupt
    // is not used in this example.  This is useful for debug purposes.
    // The shell ISR routines are found in F2837xD_DefaultIsr.c.
    // This function is found in F2837xD_PieVect.c.
    InitPieVectTable();

    // Interrupts that are used in this example are re-mapped to
    // ISR functions found within this project
    EALLOW;  // This is needed to write to EALLOW protected registers
    PieVectTable.TIMER0_INT = &cpu_timer0_isr;
    PieVectTable.TIMER1_INT = &cpu_timer1_isr;
    PieVectTable.TIMER2_INT = &cpu_timer2_isr;
    PieVectTable.SCIA_RX_INT = &RXAINT_recv_ready;
    PieVectTable.SCIB_RX_INT = &RXBINT_recv_ready;
    PieVectTable.SCIC_RX_INT = &RXCINT_recv_ready;
    PieVectTable.SCID_RX_INT = &RXDINT_recv_ready;
    PieVectTable.SCIA_TX_INT = &TXAINT_data_sent;
    PieVectTable.SCIB_TX_INT = &TXBINT_data_sent;
    PieVectTable.SCIC_TX_INT = &TXCINT_data_sent;
    PieVectTable.SCID_TX_INT = &TXDINT_data_sent;
    PieVectTable.ADCC1_INT = &ADCC_ISR;
    PieVectTable.SPIB_RX_INT = &SPIB_isr;
    PieVectTable.EMIF_ERROR_INT = &SWI1_HighestPriority;  // Using Interrupt12 interrupts that are not used as SWIs
    PieVectTable.RAM_CORRECTABLE_ERROR_INT = &SWI2_MiddlePriority;
    PieVectTable.FLASH_CORRECTABLE_ERROR_INT = &SWI3_LowestPriority;
    // ----- code for CAN start here -----
    PieVectTable.CANB0_INT = &can_isr;
    // ----- code for CAN end here -----
    EDIS;    // This is needed to disable write to EALLOW protected registers

    // Initialize the CpuTimers Device Peripheral. This function can be
    // found in F2837xD_CpuTimers.c
    InitCpuTimers();

    // Configure CPU-Timer 0, 1, and 2 to interrupt every second:
    // 200MHz CPU Freq,                     Period (in uSeconds)
    ConfigCpuTimer(&CpuTimer0, LAUNCHPAD_CPU_FREQUENCY, 10000);  // Currently not used for any purpose
    ConfigCpuTimer(&CpuTimer1, LAUNCHPAD_CPU_FREQUENCY, 100000); // !!!!! Important, Used to command LADAR every 100ms.  Do not Change.
    ConfigCpuTimer(&CpuTimer2, LAUNCHPAD_CPU_FREQUENCY, 40000); // Currently not used for any purpose

    // Enable CpuTimer Interrupt bit TIE
    CpuTimer0Regs.TCR.all = 0x4000;
    CpuTimer1Regs.TCR.all = 0x4000;
    CpuTimer2Regs.TCR.all = 0x4000;

    DELAY_US(1000000);  // Delay 1 second giving Lidar Time to power on after system power on

```

```
    init_serialSCIB(&SerialB,19200);
    init_serialSCIC(&SerialC,19200);

    for (LADARi = 0; LADARi < 228; LADARi++) {
        ladar_data[LADARi].angle = ((3*LADARi+44)*0.3515625-135)*0.01745329; //0.017453292519943 is pi/180, multiplication is faster; 0.3515625 is 360/1024
    }

    init_eQEPs();
    init_EPWM1and2();
    init_RCServoPWM_3AB_5AB_6A();
    init_ADCsAndDACs();
    setupSpib();

    EALLOW;
    EPwm4Regs.ETSEL.bit.SOCAEN = 0; // Disable SOC on A group
    EPwm4Regs.TBCTL.bit.CTRMODE = 3; // freeze counter
    EPwm4Regs.ETSEL.bit.SOCASEL = 2; // Select Event when counter equal to PRD
    EPwm4Regs.ETPS.bit.SOCAPRD = 1; // Generate pulse on 1st event
    EPwm4Regs.TBCTR = 0x0; // Clear counter
    EPwm4Regs.TBPHS.bit.TBPHS = 0x0000; // Phase is 0
    EPwm4Regs.TBCTL.bit.PHSEN = 0; // Disable phase loading
    EPwm4Regs.TBCTL.bit.CLKDIV = 0; // divide by 1  50Mhz Clock
    EPwm4Regs.TBPRD = 50000;  // Set Period to 1ms sample.  Input clock is 50MHz.
    // Notice here that we are not setting CMPA or CMPB because we are not using the PWM signal
    EPwm4Regs.ETSEL.bit.SOCAEN = 1; //enable SOCA
    //EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze,  wait to do this right before enabling interrupts
    EDIS;

    // Enable CPU int1 which is connected to CPU-Timer 0, CPU int13
    // which is connected to CPU-Timer 1, and CPU int 14, which is connected
    // to CPU-Timer 2:  int 12 is for the SWI.
    IER |= M_INT1;
    IER |= M_INT6;
    IER |= M_INT8;  // SCIC SCID
    IER |= M_INT9;  // SCIA
    IER |= M_INT12;
    IER |= M_INT13;
    IER |= M_INT14;

    // Enable TINT0 in the PIE: Group 1 interrupt 7
    PieCtrlRegs.PIEIER1.bit.INTx7 = 1;
    PieCtrlRegs.PIEIER1.bit.INTx3 = 1;
    PieCtrlRegs.PIEIER6.bit.INTx3 = 1;
    PieCtrlRegs.PIEIER12.bit.INTx9 = 1; //SWI1
    PieCtrlRegs.PIEIER12.bit.INTx10 = 1; //SWI2
    PieCtrlRegs.PIEIER12.bit.INTx11 = 1; //SWI3  Lowest priority
    // ----- code for CAN start here -----
    // Enable CANB in the PIE: Group 9 interrupt 7
    PieCtrlRegs.PIEIER9.bit.INTx7 = 1;
    // ----- code for CAN end here -----

    // ----- code for CAN start here -----
    // Enable the CAN interrupt signal
    CanbRegs.CAN_GLB_INT_EN.bit.GLBINT0_EN = 1;
    // ----- code for CAN end here -----

    // 58.5 in *  7.5 tiles = 438.75 inches = 11.14425 meters


    robotdest[0].x = 0;    robotdest[0].y = 11;
    robotdest[1].x = -8;    robotdest[1].y = 11;
    //middle of bottom
    //    robotdest[2].x = 0;     robotdest[2].y = 2;
    //    //outside the course
    //    robotdest[3].x = 0;     robotdest[3].y = -3;
    //    //back to middle
    //    robotdest[4].x = 0;     robotdest[4].y = 2;
    //    robotdest[5].x = 4;     robotdest[5].y = 2;
    //    robotdest[6].x = 4;     robotdest[6].y = 10;
    //    robotdest[7].x = 0;     robotdest[7].y = 9;
    //    // the two extra points that we added for the robot to go to after the initial course one in the middle of the field and one outside near the wall - ER
    //    robotdest[8].x = 2;     robotdest[8].y = 5;
    //    robotdest[9].x = -2;    robotdest[9].y = -1;

    // ROBOTps will be updated by Optitrack during gyro calibration
    // TODO: specify the starting position of the robot
    ROBOTps.x = 0;           //the estimate in array form (useful for matrix operations)
    ROBOTps.y = 0;
    ROBOTps.theta = 0;  // was -PI: need to flip OT ground plane to fix this
    x_pred[0][0] = ROBOTps.x; //estimate in structure form (useful elsewhere)
    x_pred[1][0] = ROBOTps.y;
    x_pred[2][0] = ROBOTps.theta;

    AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1;  //clear interrupt flag
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
    SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
    SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
    EPwm4Regs.TBCTL.bit.CTRMODE = 0; //unfreeze, and enter up count mode

    init_serialSCIA(&SerialA,115200);
    init_serialSCID(&SerialD,2083332);

    // Enable global Interrupts and higher priority real-time debug events
    EINT;  // Enable Global interrupt INTM
    ERTM;  // Enable Global realtime interrupt DBGM
    // ----- code for CAN start here -----
    // Measured Distance from 1
    // Initialize the receive message object 1 used for receiving CAN messages.
    // Message Object Parameters:
    //      Message Object ID Number: 1
    //      Message Identifier: 0x060b0101
    //      Message Frame: Standard
    //      Message Type: Receive
    //      Message ID Mask: 0x0
    //      Message Object Flags: Receive Interrupt
    //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
    //      for a Receive mailbox)
    //
    CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_1, 0x060b0101, CAN_MSG_FRAME_EXT,
                          CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
                          RX_MSG_DATA_LENGTH);

    // Measured Distance from 2
    // Initialize the receive message object 2 used for receiving CAN messages.
    // Message Object Parameters:
    //      Message Object ID Number: 2
    //      Message Identifier: 0x060b0102
    //      Message Frame: Standard
    //      Message Type: Receive
    //      Message ID Mask: 0x0
    //      Message Object Flags: Receive Interrupt
    //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
    //      for a Receive mailbox)
    //

    CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_2, 0x060b0102, CAN_MSG_FRAME_EXT,
                          CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
                          RX_MSG_DATA_LENGTH);

    // Measurement Quality from 1
    // Initialize the receive message object 2 used for receiving CAN messages.
    // Message Object Parameters:
    //      Message Object ID Number: 3
    //      Message Identifier: 0x060b0201
    //      Message Frame: Standard
    //      Message Type: Receive
    //      Message ID Mask: 0x0
    //      Message Object Flags: Receive Interrupt
    //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
    //      for a Receive mailbox)
    //
```

```
567
568     CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_3, 0x060b0201, CAN_MSG_FRAME_EXT,
569                           CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
570                           RX_MSG_DATA_LENGTH);
571
572     // Measurement Quality from 2
573     // Initialize the receive message object 2 used for receiving CAN messages.
574     // Message Object Parameters:
575     //      Message Object ID Number: 4
576     //      Message Identifier: 0x060b0202
577     //      Message Frame: Standard
578     //      Message Type: Receive
579     //      Message ID Mask: 0x0
580     //      Message Object Flags: Receive Interrupt
581     //      Message Data Length: 8 Bytes (Note that DLC field is a "don't care"
582     //      for a Receive mailbox)
583     //
584
585     CANsetupMessageObject(CANB_BASE, RX_MSG_OBJ_ID_4, 0x060b0202, CAN_MSG_FRAME_EXT,
586                           CAN_MSG_OBJ_TYPE_RX, 0, CAN_MSG_OBJ_RX_INT_ENABLE,
587                           RX_MSG_DATA_LENGTH);
588
589     //
590     // Start CAN module operations
591     //
592     CanbRegs.CAN_CTL.bit.Init = 0;
593     CanbRegs.CAN_CTL.bit.CCE = 0;
594
595
596     // ----- code for CAN end here -----
597     char S_command[19] = "S1152000124000\n";//this change the baud rate to 115200
598     uint16_t S_len = 19;
599     serial_sendSCIC(&SerialC, S_command, S_len);
600
601
602     DELAY_US(1000000);  // Delay letting Lidar change its Baud rate
603     init_serialSCIC(&SerialC,115200);
604
605
606     // LED1 is GPIO22
607     GpioDataRegs.GPACLEAR.bit.GPIO22 = 1;
608     // LED2 is GPIO94
609     GpioDataRegs.GPCCLEAR.bit.GPIO94 = 1;
610     // LED3 is GPIO95
611     GpioDataRegs.GPCCLEAR.bit.GPIO95 = 1;
612     // LED4 is GPIO97
613     GpioDataRegs.GPDCLEAR.bit.GPIO97 = 1;
614     // LED5 is GPIO111
615     GpioDataRegs.GPDCLEAR.bit.GPIO111 = 1;
616
617
618     // IDLE loop. Just sit and loop forever (optional):
619     while(1)
620     {
621         if (UARTPrint == 1 ) {
622             //UART_printfLine(1,"RCangle:%.2f",RCangle);
623             if (readbuttons() == 0) {
624                 UART_printfLine(1,"RobotState: %d", RobotState);
625                 //                UART_printfLine(1,"O1A:%.0fC:%.0fR:%.0f",MaxAreaThreshold1,MaxColThreshold1,MaxRowThreshold1);
626                 //                UART_printfLine(2,"P1A:%.0fC:%.0fR:%.0f",MaxAreaThreshold2,MaxColThreshold2,MaxRowThreshold2);
627                 //                UART_printfLine(1,"x:%.2f:y:%.2f:a%.2f",ROBOTps.x,ROBOTps.y,ROBOTps.theta);
628                 UART_printfLine(2,"orange: %.2f", robotToBall2);
629             } else if (readbuttons() == 1) {
630                 //                UART_printfLine(1,"O1A:%.0fC:%.0fR:%.0f",MaxAreaThreshold1,MaxColThreshold1,MaxRowThreshold1);
631                 //                UART_printfLine(2,"P1A:%.0fC:%.0fR:%.0f",MaxAreaThreshold2,MaxColThreshold2,MaxRowThreshold2);
632                 //UART_printfLine(1,"LV1:%.3f LV2:%.3f",printLV1,printLV2);
633                 //UART_printfLine(2,"Ln1:%.3f Ln2:%.3f",printLinux1,printLinux2);
634             } else if (readbuttons() == 2) {
635                 UART_printfLine(1,"O2A:%.0fC:%.0fR:%.0f",NextLargestAreaThreshold1,NextLargestColThreshold1,NextLargestRowThreshold1);
636                 UART_printfLine(2,"P2A:%.0fC:%.0fR:%.0f",NextLargestAreaThreshold2,NextLargestColThreshold2,NextLargestRowThreshold2);
637                 // UART_printfLine(1,"%.2f %.2f",adcC2Volt,adcC3Volt);
638                 // UART_printfLine(2,"%.2f %.2f",adcC4Volt,adcC5Volt);
639             } else if (readbuttons() == 4) {
640                 UART_printfLine(1,"O3A:%.0fC:%.0fR:%.0f",NextNextLargestAreaThreshold1,NextNextLargestColThreshold1,NextNextLargestRowThreshold1);
641                 UART_printfLine(2,"P3A:%.0fC:%.0fR:%.0f",NextNextLargestAreaThreshold2,NextNextLargestColThreshold2,NextNextLargestRowThreshold2);
642                 // UART_printfLine(1,"L:%.3f R:%.3f",LeftVel,RightVel);
643                 // UART_printfLine(2,"uL:%.2f uR:%.2f",uLeft,uRight);
644             } else if (readbuttons() == 8) {
645                 UART_printfLine(1,"020x%.2f y%.2f",ladar_pts[20].x,ladar_pts[20].y);
646                 UART_printfLine(2,"150x%.2f y%.2f",ladar_pts[150].x,ladar_pts[150].y);
647             } else if (readbuttons() == 3) {
648                 UART_printfLine(1,"Vrf:%.2f trn:%.2f",vref,turn);
649                 UART_printfLine(2,"MPU:%.2f LPR:%.2f",gyro9250_radians,gyroLPR510_radians);
650             } else if (readbuttons() == 5) {
651                 UART_printfLine(1,"Ox:%.2f:Oy:%.2f:Oa%.2f",OPTITRACKps.x,OPTITRACKps.y,OPTITRACKps.theta);
652                 UART_printfLine(2,"State:%d : %d",RobotState,statePos);
653             } else if (readbuttons() == 6) {
654                 UART_printfLine(1,"D1 %ld D2 %ld",dis_1,dis_2);
655                 UART_printfLine(2,"St1 %ld St2 %ld",measure_status_1,measure_status_2);
656             } else if (readbuttons() == 7) {
657                 UART_printfLine(1,"%.0f,%.1f,%.1f,%.1f",tagid,tagx,tagy,tagz);
658                 UART_printfLine(2,"%.1f,%.1f,%.1f",tagthetax,tagthetay,tagthetaz);
659             }
660
661             UARTPrint = 0;
662         }
663     }
664 }
665
666 __interrupt void SPIB_isr(void){
667
668     uint16_t i;
669     GpioDataRegs.GPBSET.bit.GPIO32 = 1;
670
671     GpioDataRegs.GPCSET.bit.GPIO66 = 1;                 //Pull CS high as done R/W
672     GpioDataRegs.GPASET.bit.GPIO29 = 1;  // Pull CS high for DAN28027
673
674     if (CurrentChip == MPU9250) {
675
676         for (i=0; i<8; i++) {
677             readdata[i] = SpibRegs.SPIRXBUF; // readdata[0] is garbage
678         }
679
680         PostSWI1(); // Manually cause the interrupt for the SWI1
681
682     } else if (CurrentChip == DAN28027) {
683
684         DAN28027Garbage = SpibRegs.SPIRXBUF;
685         dan28027adc1 = SpibRegs.SPIRXBUF;
686         dan28027adc2 = SpibRegs.SPIRXBUF;
687         CurrentChip = MPU9250;
688         SpibRegs.SPIFFCT.bit.TXDLY = 0;
689     }
690
691     SpibRegs.SPIFFRX.bit.RXFFOVFCLR=1; // Clear Overflow flag
692     SpibRegs.SPIFFRX.bit.RXFFINTCLR=1; // Clear Interrupt flag
693     PieCtrlRegs.PIEACK.all = PIEACK_GROUP6;
694     GpioDataRegs.GPBCLEAR.bit.GPIO32 = 1;
695 }
696
697
698 //adcd1 pie interrupt
699 __interrupt void ADCC_ISR (void)
700 {
701     GpioDataRegs.GPASET.bit.GPIO11 = 1;
702     adcc2result = AdccResultRegs.ADCRESULT0;
703     adcc3result = AdccResultRegs.ADCRESULT1;
704     adcc4result = AdccResultRegs.ADCRESULT2;
705     adcc5result = AdccResultRegs.ADCRESULT3;
706
707     // Here covert ADCIND0 to volts
708     adcC2Volt = adcc2result*3.0/4095.0;
```

```
    adcC3Volt = adcc3result*3.0/4095.0;
    adcC4Volt = adcc4result*3.0/4095.0;
    adcC5Volt = adcc5result*3.0/4095.0;

    if (MPU9250ignoreCNT >= 1) {
        CurrentChip = MPU9250;
        SpibRegs.SPIFFCT.bit.TXDLY = 0;
        SpibRegs.SPIFFRX.bit.RXFFIL = 8;

        GpioDataRegs.GPCCLEAR.bit.GPIO66 = 1;

        SpibRegs.SPITXBUF = ((0x8000)|(0x3A00));
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
        SpibRegs.SPITXBUF = 0;
    } else {
        MPU9250ignoreCNT++;
    }

    numADCCcalls++;
    AdccRegs.ADCINTFLGCLR.bit.ADCINT1 = 1;  //clear interrupt flag
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
    GpioDataRegs.GPACLEAR.bit.GPIO11 = 1;

}

// cpu_timer0_isr - CPU Timer0 ISR
__interrupt void cpu_timer0_isr(void)
{
    CpuTimer0.InterruptCount++;

    numTimer0calls++;

    if ((numTimer0calls%5) == 0) {
        // Blink LaunchPad Red LED
        //GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1;
    }

    // Acknowledge this interrupt to receive more interrupts from group 1
    PieCtrlRegs.PIEACK.all = PIEACK_GROUP1;
}

// cpu_timer1_isr - CPU Timer1 ISR
__interrupt void cpu_timer1_isr(void)
{

    serial_sendSCIC(&SerialC, G_command, G_len);

    CpuTimer1.InterruptCount++;
}

// cpu_timer2_isr CPU Timer2 ISR
__interrupt void cpu_timer2_isr(void)
{
    // Blink LaunchPad Blue LED
    //GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1;
    PostSWI3(); //Added PostSWI3 for Lab 7 Ex.1 Step 1 - DS
    CpuTimer2.InterruptCount++;

    //  if ((CpuTimer2.InterruptCount % 10) == 0) {
    //      UARTPrint = 1;
    //  }
}

void setF28027EPWM1A(float controleffort){
    if (controleffort < -10) {
        controleffort = -10;
    }
    if (controleffort > 10) {
        controleffort = 10;
    }
    float value = (controleffort+10)*3000.0/20.0;
    EPwm1A_F28027 = (int16_t)value;  // Set global variable that is sent over SPI to F28027
}
void setF28027EPWM2A(float controleffort){
    if (controleffort < -10) {
        controleffort = -10;
    }
    if (controleffort > 10) {
        controleffort = 10;
    }
    float value = (controleffort+10)*3000.0/20.0;
    EPwm2A_F28027 = (int16_t)value;  // Set global variable that is sent over SPI to F28027
}

//
// Connected to PIEIER12_9 (use MINT12 and MG12_9 masks):
//
__interrupt void SWI1_HighestPriority(void)     // EMIF_ERROR
{
    GpioDataRegs.GPBSET.bit.GPIO61 = 1;
    // Set interrupt priority:
    volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
    IER |= M_INT12;
    IER    &= MINT12;                           // Set "global" priority
    PieCtrlRegs.PIEIER12.all &= MG12_9;  // Set "group"  priority
    PieCtrlRegs.PIEACK.all = 0xFFFF;    // Enable PIE interrupts
    __asm("  NOP");
    EINT;

    //#############################################################################################################
    IMU_data[0] = readdata[1];
    IMU_data[1] = readdata[2];
    IMU_data[2] = readdata[3];
    IMU_data[3] = readdata[5];
    IMU_data[4] = readdata[6];
    IMU_data[5] = readdata[7];

    accelx = (((float)(IMU_data[0]))*4.0/32767.0);
    accely = (((float)(IMU_data[1]))*4.0/32767.0);
    accelz = (((float)(IMU_data[2]))*4.0/32767.0);
    gyrox  = (((float)(IMU_data[3]))*250.0/32767.0);
    gyroy  = (((float)(IMU_data[4]))*250.0/32767.0);
    gyroz  = (((float)(IMU_data[5]))*250.0/32767.0);

    if(calibration_state == 0){
        calibration_count++;
        if (calibration_count == 2000) {
            calibration_state = 1;
            calibration_count = 0;
        }
    } else if(calibration_state == 1){
        accelx_offset+=accelx;
        accely_offset+=accely;
        accelz_offset+=accelz;
        gyrox_offset+=gyrox;
        gyroy_offset+=gyroy;
        gyroz_offset+=gyroz;
        gyroLPR510_offset+=adcC5Volt;
        calibration_count++;
        if (calibration_count == 2000) {
            calibration_state = 2;
            accelx_offset/=2000.0;
            accely_offset/=2000.0;
            accelz_offset/=2000.0;
            gyrox_offset/=2000.0;
            gyroy_offset/=2000.0;
            gyroz_offset/=2000.0;
```

```
            gyroLPR510_offset/=2000.0;
            calibration_count = 0;
        }
    } else if(calibration_state == 2) {

        gyroLPR510_drift += (((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005)/(2000);
        old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;

        gyro9250_drift += (((gyroz-gyroz_offset) + old_gyro9250)*.0005)/(2000);
        old_gyro9250 = gyroz-gyroz_offset;
        calibration_count++;

        if (calibration_count == 2000) {
            calibration_state = 3;
            calibration_count = 0;
            doneCal = 1;
            newOPTITRACKpose = 0;
        }
    } else if(calibration_state == 3){
        accelx -=(accelx_offset);
        accely -=(accely_offset);
        accelz -=(accelz_offset);
        gyrox -= gyrox_offset;
        gyroy -= gyroy_offset;
        gyroz -= gyroz_offset;
        LeftWheel = readEncLeft();
        RightWheel = readEncRight();
        HandValue = readEncWheel();

        gyro9250_angle = gyro9250_angle + (gyroz + old_gyro9250)*.0005 - gyro9250_drift;
        old_gyro9250 = gyroz;

        gyroLPR510_angle = gyroLPR510_angle + ((adcC5Volt-gyroLPR510_offset) + old_gyroLPR510)*.0005 - gyroLPR510_drift;
        old_gyroLPR510 = adcC5Volt-gyroLPR510_offset;

        gyro9250_radians = (gyro9250_angle * (PI/180.0));
        gyroLPR510_radians = gyroLPR510_angle * 400 * (PI/180.0);

        LeftVel = (1.235/12.0)*(LeftWheel - LeftWheel_1)*1000;
        RightVel = (1.235/12.0)*(RightWheel - RightWheel_1)*1000;

        if (newLinuxCommands == 1) {
            newLinuxCommands = 0;
            printLinux1 = LinuxCommands[0];
            printLinux2 = LinuxCommands[1];
            //?? = LinuxCommands[2];
            //?? = LinuxCommands[3];
            //?? = LinuxCommands[4];
            //?? = LinuxCommands[5];
            //?? = LinuxCommands[6];
            //?? = LinuxCommands[7];
            //?? = LinuxCommands[8];
            //?? = LinuxCommands[9];
            //?? = LinuxCommands[10];
        }

        if (NewCAMDataThreshold1 == 1) {
            NewCAMDataThreshold1 = 0;
            MaxAreaThreshold1 = fromCAMvaluesThreshold1[0];
            MaxColThreshold1 = fromCAMvaluesThreshold1[1];
            MaxRowThreshold1 = fromCAMvaluesThreshold1[2];

            // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
            robotToBall1 = -0.00002154*MaxRowThreshold1*MaxRowThreshold1*MaxRowThreshold1 + 0.006486*MaxRowThreshold1*MaxRowThreshold1 - 0.6728*MaxRowThreshold1 + 25.42;

            NextLargestAreaThreshold1 = fromCAMvaluesThreshold1[3];
            NextLargestColThreshold1 = fromCAMvaluesThreshold1[4];
            NextLargestRowThreshold1 = fromCAMvaluesThreshold1[5];

            NextNextLargestAreaThreshold1 = fromCAMvaluesThreshold1[6];
            NextNextLargestColThreshold1 = fromCAMvaluesThreshold1[7];
            NextNextLargestRowThreshold1 = fromCAMvaluesThreshold1[8];
            numThres1++;
            if ((numThres1 % 5) == 0) {
                // LED4 is GPIO97
                GpioDataRegs.GPDTOGGLE.bit.GPIO97 = 1;
            }
        }

        if (NewCAMDataThreshold2 == 1) {
            NewCAMDataThreshold2 = 0;
            MaxAreaThreshold2 = fromCAMvaluesThreshold2[0];
            MaxColThreshold2 = fromCAMvaluesThreshold2[1];
            MaxRowThreshold2 = fromCAMvaluesThreshold2[2];
            // using the calibration values and the line of best fit found from the distance of the green golf ball in relation to the real world, this equation calculates the distance the robot is
            robotToBall2 = -0.00002154*MaxRowThreshold2*MaxRowThreshold2*MaxRowThreshold2 + 0.006486*MaxRowThreshold2*MaxRowThreshold2 - 0.6728*MaxRowThreshold2 + 25.42;

            NextLargestAreaThreshold2 = fromCAMvaluesThreshold2[3];
            NextLargestColThreshold2 = fromCAMvaluesThreshold2[4];
            NextLargestRowThreshold2 = fromCAMvaluesThreshold2[5];

            NextNextLargestAreaThreshold2 = fromCAMvaluesThreshold2[6];
            NextNextLargestColThreshold2 = fromCAMvaluesThreshold2[7];
            NextNextLargestRowThreshold2 = fromCAMvaluesThreshold2[8];
            numThres2++;
            if ((numThres2 % 5) == 0) {
                // LED5 is GPIO111
                GpioDataRegs.GPDTOGGLE.bit.GPIO111 = 1;
            }
        }

        if (NewCAMDataAprilTag1 == 1) {
            NewCAMDataAprilTag1 = 0;
            tagx = fromCAMvaluesAprilTag1[0];
            tagy = fromCAMvaluesAprilTag1[1];
            tagz = fromCAMvaluesAprilTag1[2];

            tagthetax = fromCAMvaluesAprilTag1[3];
            tagthetay = fromCAMvaluesAprilTag1[4];
            tagthetaz = fromCAMvaluesAprilTag1[5];

            tagid = fromCAMvaluesAprilTag1[6];

            numtag++;
            if ((numtag % 5) == 0) {
                // LED2 is GPIO94
                GpioDataRegs.GPCTOGGLE.bit.GPIO94 = 1;
            }
        }

        if (NewLVData == 1) {
            NewLVData = 0;
            printLV1 = fromLVvalues[0];
            printLV2 = fromLVvalues[1];
            //?? = fromLVvalues[2];
            //?? = fromLVvalues[3];
            //?? = fromLVvalues[4];
            //?? = fromLVvalues[5];
            //?? = fromLVvalues[6];
            //?? = fromLVvalues[7];
        }


        if (new_optitrack == 1) {
            OPTITRACKps = UpdateOptitrackStates(ROBOTps, &newOPTITRACKpose);
            new_optitrack = 0;
        }

        // Step 0: update B, u
        B[0][0] = cosf(ROBOTps.theta)*0.001;
        B[1][0] = sinf(ROBOTps.theta)*0.001;
        B[2][1] = 0.001;
```

```
        u[0][0] = 0.5*(LeftVel + RightVel);    // linear velocity of robot
        u[1][0] = gyroz*(PI/180.0);    // angular velocity in rad/s (negative for right hand angle)

        // Step 1: predict the state and estimate covariance
        Matrix3x2_Mult(B, u, Bu);                   // Bu = B*u
        Matrix3x1_Add(x_pred, Bu, x_pred, 1.0, 1.0); // x_pred = x_pred(old) + Bu
        Matrix3x3_Add(P_pred, Q, P_pred, 1.0, 1.0); // P_pred = P_pred(old) + Q
        // Step 2: if there is a new measurement, then update the state
        if (1 == newOPTITRACKpose) {
            OptiNumUpdatesProcessed++;  // checking how often OptiTrack is causing a Kalman update
            if ((OptiNumUpdatesProcessed%10)==0) {
                // LED 3
                GpioDataRegs.GPCTOGGLE.bit.GPIO95 = 1;
            }
            newOPTITRACKpose = 0;
            z[0][0] = OPTITRACKps.x;    // take in the Optitrack Motion Capture measurement
            z[1][0] = OPTITRACKps.y;
            // fix for OptiTrack problem at 180 degrees
            if (cosf(ROBOTps.theta) < -0.99) {
                z[2][0] = ROBOTps.theta;
            }
            else {
                z[2][0] = OPTITRACKps.theta;
            }
            // Step 2a: calculate the innovation/measurement residual, ytilde
            Matrix3x1_Add(z, x_pred, ytilde, 1.0, -1.0);    // ytilde = z-x_pred
            // Step 2b: calculate innovation covariance, S
            Matrix3x3_Add(P_pred, R, S, 1.0, 1.0);                          // S = P_pred + R
            // Step 2c: calculate the optimal Kalman gain, K
            Matrix3x3_Invert(S, S_inv);
            Matrix3x3_Mult(P_pred,  S_inv, K);                              // K = P_pred*(S^-1)
            // Step 2d: update the state estimate x_pred = x_pred(old) + K*ytilde
            Matrix3x1_Mult(K, ytilde, temp_3x1);
            Matrix3x1_Add(x_pred, temp_3x1, x_pred, 1.0, 1.0);
            // Step 2e: update the covariance estimate   P_pred = (I-K)*P_pred(old)
            Matrix3x3_Add(eye3, K, temp_3x3, 1.0, -1.0);
            Matrix3x3_Mult(temp_3x3, P_pred, P_pred);
        }   // end of correction step

        // set ROBOTps to the updated and corrected Kalman values.
        ROBOTps.x = x_pred[0][0];
        ROBOTps.y = x_pred[1][0];
        ROBOTps.theta = x_pred[2][0];

        // Make sure this function is called every time in this function even if you decide not to use its vref and turn
        // uses xy code to step through an array of positions
        if( xy_control(&vref, &turn, 1.0, ROBOTps.x, ROBOTps.y, robotdest[statePos].x, robotdest[statePos].y, ROBOTps.theta, 0.25, 0.5)) {
            statePos = (statePos+1)%NUMWAYPOINTS;
        }

        dwell++;
        // state machine
        colcentroid1 = MaxColThreshold1 - 80;
        colcentroid2 = MaxColThreshold2 - 80; // 80 is used as the center column of the camera, so a centered green or orange ball centered gives concentroid = 0 - DS
        switch (RobotState) {
        case 1: // command the robot to an X,Y point in the course - KL
            // vref and turn are the vref and turn returned from xy_control
            if (LADARfront < 1.2) {
                vref = 0.2;
                checkfronttally++;
                if (checkfronttally > 310) { // check if LADARfront < 1.2 for 310ms or 3 LADAR samples
                    RobotState = 10; // Wall follow
                    WallFollowtime = 0;
                    right_wall_follow_state = 1;
                }
            } else {
                checkfronttally = 0;
            }
            // ensures that the robot does not move for 2 seconds after returning to state 1, so after is stops detecting a golf ball - ER
            statecount++;
            if (statecount<=2000){
                RobotState = 1;
            }else{
                if (MaxAreaThreshold1 > 30) { // green ball detection threshold chosen so camera noise (image disturbance, not literal) does not trigger chasing rolling ball behavior - DS
                    RobotState = 20;
                    statecount =0;
                }
                if (MaxAreaThreshold2 > 30) { // same detection threshold but for orange ball - DS
                    RobotState = 30;
                    statecount =0;
                }
                if (tagx <= 0.0993) {
                    if (tagid == 0.0){
                        RobotState = 40;
                    }
                    if (tagid == 1.0){
                        RobotState = 50;
                    }
                }

            }
            break;
        case 10: // right wall following if an obstacle gets in the way of robot - KL
            if (right_wall_follow_state == 1) {
                //Left Turn
                turn = Kp_front_wall*(14.5 - LADARfront);
                vref = front_turn_velocity;
                if (LADARfront > left_turn_Stop_threshold) {
                    right_wall_follow_state = 2;
                }
            } else if (right_wall_follow_state == 2) {
                //Right Wall Follow
                turn = Kp_right_wal*(ref_right_wall - LADARrightfront);
                vref = foward_velocity;
                if (LADARfront < left_turn_Start_threshold) {
                    right_wall_follow_state = 1;
                }
            }
            if (turn > turn_saturation) {
                turn = turn_saturation;
            }
            if (turn < -turn_saturation) {
                turn = -turn_saturation;
            }

            WallFollowtime++;
            // exit wall following if at least 5 seconds have passed and front path is clear - KL
            if ( (WallFollowtime > 5000) && (LADARfront > 1.5) ) {
                RobotState = 1; //return to XY waypoint navigation - KL
                checkfronttally = 0;
            }
            break;
            // this state sets vref and turn so that the robot moves towards a detected green golf ball and after 1 second of waiting, sends the robot to state 22 which stops the robot - ER
        case 20:
            // put vision code here
            if (MaxColThreshold1 == 0 || MaxAreaThreshold1 < 3){
                vref = 0;
                turn = 0;
            } else {
                vref = 0.75;
                turn = kpvision * (0 - colcentroid1);
                if (MaxRowThreshold1 > 100){ //KLEC: row number for rb to know how to close it is to the ball, need to tune the number for enough clearance so that gate doesn't push ball away
                    RobotState = 22;
                    count = 0;
                }
            }
            break;
        case 22: // after no longer detecting the green golf ball, the robot stops moving - ER, lift arm and move tongue
            vref = 0;
            turn = 0;

            count++;
```

```
            if (count>=1000){//after 1 second, robot drives forward - KL
                RobotState = 24;
                count = 0;
            }
            break;
        case 24: // robot drives forward into the golf ball - KL
            vref = 0.5;
            turn = 0;
            setEPWM5B_RCServo(LTong); //KLEC: move the tongue to the left side, green balls go to the right compartment
            setEPWM6A_RCServo(Gate_O); //KLEC: open the gate
            count++;
            if (count>=2000){
                RobotState = 26;
                count = 0;
            }
            break;
        case 26: // robot pauses once ball is presumed to be captured - KL
            vref = 0;
            turn = 0;
            setEPWM6A_RCServo(Gate_C);
            setEPWM5B_RCServo(MTong);
            count++;
            if (count>=1000){ //after 1 second, robot returns to waypoint navigation - KL
                RobotState = 1;
                count = 0;
                statecount = 0;
            }
            break;

            // states for orange ball
        case 30:
            // put vision code here
            //saying that if the orange ball is not detected or doesn't fit in the area threshold, then to stop the robot - KP
            //this helps it not detect other orange objects in the room as the ball - KP
            if (MaxColThreshold2 == 0 || MaxAreaThreshold2 < 3){
                vref = 0;
                turn = 0;
            } else {
                //tells it what to do if it is detected, move to state 32 - KP
                vref = 0.75;
                turn = kpvision * (0 - colcentroid2);
                if (MaxRowThreshold2 > 100){
                    RobotState = 32;
                    count = 0;
                }
            }
            break;
        case 32:
            //stopping and pausing with vref and turn at 0 - KP
            vref = 0;
            turn = 0;
            setEPWM5B_RCServo(RTong); //KLEC: move the tongue to the right side, orange balls go to the left compartment
            setEPWM6A_RCServo(Gate_O); //KLEC: open the gate
            count++;
            if (count>=1000){
                RobotState = 34;
                count = 0;
            }
            break;
        case 34:
            //telling robot to move forward then move to state 36 - KP
            vref = 0.5;
            turn = 0;
            setEPWM5B_RCServo(RTong);
            setEPWM6A_RCServo(Gate_O);
            count++;
            if (count>=1000){
                RobotState = 36;
                count = 0;
            }
            break;
        case 36:
            //another stop before it begins the route again - KP
            vref = 0;
            turn = 0;
            setEPWM6A_RCServo(Gate_C);
            setEPWM5B_RCServo(MTong);
            count++;
            if (count>=1000){
                RobotState = 1;
                count = 0;
                statecount = 0;
            }
            break;
        case 40://code telling the gate to open when it detects april tag 0
            vref = 0;
            turn = 0;

            setEPWM6A_RCServo(Gate_O);
            setEPWM5B_RCServo(0);

            if (dwell >= 2000){
                setEPWM6A_RCServo(Gate_C);
            }

            count++;
            if (count>=1000){
                RobotState = 1;
                count = 0;
                statecount = 0;
            }
            break;
        case 50: //code telling the robot to turn

            vref = 0;
            count++;
            if (count>=2000){
                RobotState = 52;
                count = 0;
            }
            break;
        case 52:
            current = ROBOTps.theta;
            target = PI;
            turn = fabs(target - current);
            vref = 1;
            count++;
            if (count>=1000){
                RobotState = 1;
                count = 0;
                statecount = 0;
            }
            break;

        default:
            break;
        }


        //Must be called each time into this SWI1 function
        PIcontrol(&uLeft,&uRight,vref,turn,LeftWheel,RightWheel);
        // These below lines also must be called each time into this SWI1 function
        setEPWM1A(uLeft);
        setEPWM2A(-uRight);
        setF28027EPWM1A(uLeft);
        setF28027EPWM2A(-uRight);
        LeftWheel_1 = LeftWheel;
        RightWheel_1 = RightWheel;

        if((timecount%250) == 0) {
            DataToLabView.floatData[0] = ROBOTps.x;
            DataToLabView.floatData[1] = ROBOTps.y;
```

```
            DataToLabView.floatData[2] = ROBOTps.theta;
            DataToLabView.floatData[3] = (float)timecount;
            DataToLabView.floatData[4] = 0.5*(LeftVel + RightVel);
            DataToLabView.floatData[5] = (float)RobotState;
            DataToLabView.floatData[6] = (float)statePos;
            DataToLabView.floatData[7] = LADARfront;
            LVsenddata[0] = '*';  // header for LVdata
            LVsenddata[1] = '$';
            for (i=0;i<LVNUM_TOFROM_FLOATS*4;i++) {
                if (i%2==0) {
                    LVsenddata[i+2] = DataToLabView.rawData[i/2] & 0xFF;
                } else {
                    LVsenddata[i+2] = (DataToLabView.rawData[i/2]>>8) & 0xFF;
                }
            }
            serial_sendSCID(&SerialD, LVsenddata, 4*LVNUM_TOFROM_FLOATS + 2);
        }

    }
    timecount++;
    if((timecount%200) == 0)
    {
        if(doneCal == 0) {
            GpioDataRegs.GPATOGGLE.bit.GPIO31 = 1; // Blink Blue LED while calibrating
        }
        GpioDataRegs.GPBTOGGLE.bit.GPIO34 = 1; // Always Block Red LED
        UARTPrint = 1; // Tell While loop to print
    }

    SpibRegs.SPIFFCT.bit.TXDLY = 16;
    CurrentChip = DAN28027;
    SpibRegs.SPIFFRX.bit.RXFFIL = 3;
    GpioDataRegs.GPACLEAR.bit.GPIO29 = 1;
    SpibRegs.SPITXBUF = 0x00DA;
    SpibRegs.SPITXBUF = EPwm1A_F28027;
    SpibRegs.SPITXBUF = EPwm2A_F28027;

    //###########################################################################################################
    //
    // Restore registers saved:
    //
    DINT;
    PieCtrlRegs.PIEIER12.all = TempPIEIER;

    GpioDataRegs.GPBCLEAR.bit.GPIO61 = 1;
}

//
// Connected to PIEIER12_10 (use MINT12 and MG12_10 masks):
//
__interrupt void SWI2_MiddlePriority(void)     // RAM_CORRECTABLE_ERROR
{
    // Set interrupt priority:
    volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
    IER |= M_INT12;
    IER    &= MINT12;                           // Set "global" priority
    PieCtrlRegs.PIEIER12.all &= MG12_10;  // Set "group"  priority
    PieCtrlRegs.PIEACK.all = 0xFFFF;    // Enable PIE interrupts
    __asm("  NOP");
    EINT;

    //###########################################################################################
    // Insert SWI ISR Code here.......
    // LED1
    GpioDataRegs.GPATOGGLE.bit.GPIO22 = 1;
    if (LADARpingpong == 1) {
        // LADARrightfront is the min of dist 52, 53, 54, 55, 56
        LADARrightfront = 19; // 19 is greater than max feet
        for (LADARi = 52; LADARi <= 56 ; LADARi++) {
            if (ladar_data[LADARi].distance_ping < LADARrightfront) {
                LADARrightfront = ladar_data[LADARi].distance_ping;
            }
        }
        // LADARfront is the min of dist 111, 112, 113, 114, 115
        LADARfront = 19;
        for (LADARi = 111; LADARi <= 115 ; LADARi++) {
            if (ladar_data[LADARi].distance_ping < LADARfront) {
                LADARfront = ladar_data[LADARi].distance_ping;
            }
        }
        LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
        LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
        for (LADARi = 0; LADARi < 228; LADARi++) {

            ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_ping*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
            ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_ping*sinf(ladar_data[LADARi].angle + ROBOTps.theta);

        }
    } else if (LADARpingpong == 0) {
        // LADARrightfront is the min of dist 52, 53, 54, 55, 56
        LADARrightfront = 19; // 19 is greater than max feet
        for (LADARi = 52; LADARi <= 56 ; LADARi++) {
            if (ladar_data[LADARi].distance_pong < LADARrightfront) {
                LADARrightfront = ladar_data[LADARi].distance_pong;
            }
        }
        // LADARfront is the min of dist 111, 112, 113, 114, 115
        LADARfront = 19;
        for (LADARi = 111; LADARi <= 115 ; LADARi++) {
            if (ladar_data[LADARi].distance_pong < LADARfront) {
                LADARfront = ladar_data[LADARi].distance_pong;
            }
        }
        LADARxoffset = ROBOTps.x + (LADARps.x*cosf(ROBOTps.theta)-LADARps.y*sinf(ROBOTps.theta - PI/2.0));
        LADARyoffset = ROBOTps.y + (LADARps.x*sinf(ROBOTps.theta)-LADARps.y*cosf(ROBOTps.theta - PI/2.0));
        for (LADARi = 0; LADARi < 228; LADARi++) {

            ladar_pts[LADARi].x = LADARxoffset + ladar_data[LADARi].distance_pong*cosf(ladar_data[LADARi].angle + ROBOTps.theta);
            ladar_pts[LADARi].y = LADARyoffset + ladar_data[LADARi].distance_pong*sinf(ladar_data[LADARi].angle + ROBOTps.theta);

        }
    }


    //###########################################################################################
    //
    // Restore registers saved:
    //
    DINT;
    PieCtrlRegs.PIEIER12.all = TempPIEIER;

}

//
// Connected to PIEIER12_11 (use MINT12 and MG12_11 masks):
//
__interrupt void SWI3_LowestPriority(void)     // FLASH_CORRECTABLE_ERROR
{
    // Set interrupt priority:
    volatile Uint16 TempPIEIER = PieCtrlRegs.PIEIER12.all;
    IER |= M_INT12;
    IER    &= MINT12;                           // Set "global" priority
    PieCtrlRegs.PIEIER12.all &= MG12_11;  // Set "group"  priority
    PieCtrlRegs.PIEACK.all = 0xFFFF;    // Enable PIE interrupts
    __asm("  NOP");
    EINT;
    RCangle = readEncWheel(); //Lab RC servo exercise uses encoder angle as the command input for servo motion - DS
    //    setEPWM3A_RCServo(RCangle); //RCangle is a value from -90 to 90
    //    setEPWM3B_RCServo(RCangle); //RCangle is a value from -90 to 90
    //    setEPWM5A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use
    setEPWM5B_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Tongue
    setEPWM6A_RCServo(RCangle); //RCangle is a value from -90 to 90 in use - Gate
```

```
    //#############################################################################################
    // Insert SWI ISR Code here.......

    //#############################################################################################
    //
    // Restore registers saved:
    //
    DINT;
    PieCtrlRegs.PIEIER12.all = TempPIEIER;

}

// ----- code for CAN start here -----
__interrupt void can_isr(void)
{
    int i = 0;

    uint32_t status;

    //
    // Read the CAN interrupt status to find the cause of the interrupt
    //
    status = CANgetInterruptCause(CANB_BASE);

    //
    // If the cause is a controller status interrupt, then get the status
    //
    if(status == CAN_INT_INT0ID_STATUS)
    {
        //
        // Read the controller status.  This will return a field of status
        // error bits that can indicate various errors.  Error processing
        // is not done in this example for simplicity.  Refer to the
        // API documentation for details about the error status bits.
        // The act of reading this status will clear the interrupt.
        //
        status = CANgetStatus(CANB_BASE);

    }

    //
    // Check if the cause is the receive message object 2
    //
    else if(status == RX_MSG_OBJ_ID_1)
    {
        //
        // Get the received message
        //
        CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_1, rxMsgData);

        for(i = 0; i<2; i++)
        {
            dis_raw_1[i] = rxMsgData[i];
        }

        dis_1 = 256*dis_raw_1[1] + dis_raw_1[0];

        measure_status_1 = rxMsgData[2];

        //
        // Getting to this point means that the RX interrupt occurred on
        // message object 2, and the message RX is complete.  Clear the
        // message object interrupt.
        //
        CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_1);

        //
        // Increment a counter to keep track of how many messages have been
        // received. In a real application this could be used to set flags to
        // indicate when a message is received.
        //
        rxMsgCount_1++;

        //
        // Since the message was received, clear any error flags.
        //
        errorFlag = 0;
    }

    else if(status == RX_MSG_OBJ_ID_2)
    {
        //
        // Get the received message
        //
        CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_2, rxMsgData);

        for(i = 0; i<2; i++)
        {
            dis_raw_2[i] = rxMsgData[i];
        }

        dis_2 = 256*dis_raw_2[1] + dis_raw_2[0];

        measure_status_2 = rxMsgData[2];

        //
        // Getting to this point means that the RX interrupt occurred on
        // message object 2, and the message RX is complete.  Clear the
        // message object interrupt.
        //
        CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_2);

        //
        // Increment a counter to keep track of how many messages have been
        // received. In a real application this could be used to set flags to
        // indicate when a message is received.
        //
        rxMsgCount_2++;

        //
        // Since the message was received, clear any error flags.
        //
        errorFlag = 0;
    }

    else if(status == RX_MSG_OBJ_ID_3)
    {
        //
        // Get the received message
        //
        CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_3, rxMsgData);

        for(i = 0; i<4; i++)
        {
            lightlevel_raw_1[i] = rxMsgData[i];
            quality_raw_1[i] = rxMsgData[i+4];

        }

        lightlevel_1 = ((256.0*256.0*256.0)*lightlevel_raw_1[3] + (256.0*256.0)*lightlevel_raw_1[2] + 256.0*lightlevel_raw_1[1] + lightlevel_raw_1[0])/65535;
        quality_1 = ((256.0*256.0*256.0)*quality_raw_1[3] + (256.0*256.0)*quality_raw_1[2] + 256.0*quality_raw_1[1] + quality_raw_1[0])/65535;


        //
        // Getting to this point means that the RX interrupt occurred on
        // message object 2, and the message RX is complete.  Clear the
        // message object interrupt.
        //
        CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_3);

        //
        // Since the message was received, clear any error flags.
```

```
        //
        errorFlag = 0;
    }


    else if(status == RX_MSG_OBJ_ID_4)
    {
        //
        // Get the received message
        //
        CANreadMessage(CANB_BASE, RX_MSG_OBJ_ID_4, rxMsgData);

        for(i = 0; i<4; i++)
        {
            lightlevel_raw_2[i] = rxMsgData[i];
            quality_raw_2[i] = rxMsgData[i+4];

        }

        lightlevel_2 = ((256.0*256.0*256.0)*lightlevel_raw_2[3] + (256.0*256.0)*lightlevel_raw_2[2] + 256.0*lightlevel_raw_2[1] + lightlevel_raw_2[0])/65535;
        quality_2 = ((256.0*256.0*256.0)*quality_raw_2[3] + (256.0*256.0)*quality_raw_2[2] + 256.0*quality_raw_2[1] + quality_raw_2[0])/65535;

        //
        // Getting to this point means that the RX interrupt occurred on
        // message object 2, and the message RX is complete.  Clear the
        // message object interrupt.
        //
        CANclearInterruptStatus(CANB_BASE, RX_MSG_OBJ_ID_4);

        //
        // Since the message was received, clear any error flags.
        //
        errorFlag = 0;
    }



    //
    // If something unexpected caused the interrupt, this would handle it.
    //
    else
    {
        //
        // Spurious interrupt handling can go here.
        //
    }

    //
    // Clear the global interrupt flag for the CAN interrupt line
    //
    CANclearGlobalInterruptStatus(CANB_BASE, CAN_GLOBAL_INT_CANINT0);

    //
    // Acknowledge this interrupt located in group 9
    //
    InterruptclearACKGroup(INTERRUPT_ACK_GROUP9);
}
// ----- code for CAN end here -----
```